

Spring-Security

¿Qué es Spring Security?

Es el framework de seguridad estándar para aplicaciones de Java. Es el que nos explica dos preguntas fundamentales antes de permitir que cualquier petición llegue a la lógica de un negocio:

Autenticación (401 unauthorized)- ¿Quién eres? Verificar la identidad del usuario

Autorización (403 forbidden)- Ya sé quien eres, ¿tienes los permisos necesarios?

HTTP Basic

¿Qué es HTTP Basic y qué problema resuelve?

Cuando tenemos una API sin seguridad nuestros endpoints se encuentran expuestos lo que quiere decir que cualquier persona con la URL puede modificar e incluso borrar nuestros registros. Con Spring Security resolvemos esto, separando dos barreras antes de que se toque a nuestro controlador, las que mencionamos: Autenticación y Autorización. Http Basic es un mecanismo, el más simple para poder demostrar la identidad que buscamos. En cada petición, el cliente envía a su usuario y contraseña concatenados y codificados en Base64 dentro de Authorization. El servidor no guardará la memoria de sesiones pasadas (stateless) y al momento de volver a ingresar se tiene que validar de nuevo.

¿Dónde se ve en mi código?

Toda la configuración se encuentra en `src/main/java/com/luv2code/springboot/cruddemo/security/SecurityConfig.java` lo que nos asegura que el controlador original que es `EmployeeRestController` no se entera de la seguridad.

Autenticación (userService): Utilizo `JdbcUserDetailsService` para enseñarle a Spring cómo podemos buscar usuarios en mis tablas (members y roles) esto es usando consultas de SQL directas, pues Spring no sabe el esquema de nuestra base de datos.

Autorización (filterChain): Definí las reglas de acceso por verbo HTTP. Para leer (GET) nos da con el rol de `EMPLOYEE`, para modificar (POST,PUT) se requiere del rol `MANAGER` y para eliminar (DELETE) pedimos que solo `ADMIN` pueda hacerlo.

Configuración Stateless: Cuando usamos `http.sessionManagement(sesión -> session.sessionCreationPolicy.STATELESS)` se desactivan las sesiones de la memoria y también apagamos csrf con `http.csrf(csrf -> csrf.disable())` porque como no tenemos cookies de sesión en API Rest no hay peligro de falsificación de peticiones entre sitios.

¿Qué pasa si no lo utilizo?

Si no utilizamos este mecanismo entonces un comando como `curl -X DELETE http://localhost:8071/api/employees/1` borraría a nuestro empleado. Como ya tenemos la cadena de filtros y nuestra barrera ahora ese comando sin las credenciales rebotaría el 401, incluso si el usuario se autentica con un rol que no sea autorizado va a rebotar el 403.

Aunque sí, esta es la forma más básica, por lo que yo no la recomendaría o no es lo que prefiero en mis sistemas pues, aunque se protege a nuestra API, si no se usa bajo un https somos aun muy vulnerables. La cadena que mencioné de BASE64 puede ser revertida y dejamos a nuestros usuarios expuestos en texto plano en cada petición.

Hashing BCrypt

Tenemos la contraseña original de nuestro usuarios (test123) que no se guarda en nuestra base de datos. En la columna pw de la tabla de mmebers, utilizo el algoritmo hashin BCrypt, este es lento y con esto frustra los ataques brutos e inyecta un salt (o cadena aleatoria) única a cada hash. Ahora si dos usuarios tienen la misma contraseña nuestros hashes van a ser distintos gracias al prefijo que tenemos como {bcrypt} al inicio de cada hash en la bd.

Cómo correrlo

Desde nuestra terminal ingresamos a la carpeta del proyecto 01-security-basic:

Levantamos el proyecto: `./mvnw spring-boot:run`

Hacemos peticiones para verificar los permisos de los roles:

401: `curl -i http://localhost:8071/api/employess`

403: `curl -u john:test123 -X DELETE http://localhost:8071/api/employees/1`

Y lectura exitosa para el rol que tiene John: `curl -u john:test123 http://localhost:8071/api/employees/1`

Jason Web Token

¿Qué es y qué problema resuelve?

JWT está para cubrir el problema que mencioné en http basic, el que el usuario y la contraseña viajen en cada petición. Con JWT esto se resuelve cambiando la prueba de identidad: el usuario va a enviar sus credenciales una sola vez (justo en el endpoint de login) y va a recibir un token criptográfico con tiempo de caducidad, este token es el que se envía en las siguientes peticiones en vez de la información del usuario.

¿Dónde se ve en mi código?

La mayor diferencia que tenemos es en el SecurityConfig.java. ahora tenemos dos cadenas de filtros (loginFilterChain y apiFilterChain).

loginFilterChain: como tenemos @order(1) y solo se escucha /api/auth/**. Este usa HTTP Basic para validar la contraseña en la base de datos si esta es valida entonces el AuthController.java nos va a generar el token del usuario.

apiFilterChain: Tiene @Order(2) y esto protege nuestros endpoints de /api/employees. Ya no acepta contraseñas sino que usa OAuth2ResourceServer para pedir un token Bearer, en esto la autenticación es similar a las de http basic.

Firmas asimétricas (RSA): igual en nuestro SecurityConfig.java, el bean jwtEncoder utiliza una llave privada que es la private.pem para firmar los tokens que se emiten. El jwtDecoder utiliza la llave pública de public.pem para validar que nuestros tokens que entran sean auténticos.

¿Qué pasa si no se utiliza?

Si nos quedamos solo con HTTP Basic la contraseña del usuario viaja con cada una de las peticiones, JWT elimina ese riesgo haciendo que la contraseña del usuario solo viaje una vez.

Cómo correrlo

Desde la terminal en la carpeta 02-security-jwt:

Levantar el proyecto: ./mvnw spring-boot:run

Generar el token: curl -s -u john:test123 -X POST http://localhost:8072/api/auth/login

Usar el token: Copia el accessToken recibido y ejecuta curl -H "Authorization: Bearer <TU_TOKEN>" http://localhost:8072/api/employees

OAuth2

¿Qué es y qué problema resuelve?

Si con JWT resolvimos que la contraseña no viajara en cada petición, con OAuth2 resolvemos el problema de tener que gestionar esas contraseñas nosotros mismos. Con este mecanismo delegamos la identidad a un tercero de confianza (un Servidor de Autorización, en este caso Keycloak, pero podría ser Google, GitHub o Auth0).

El usuario se autentica directamente con ellos, y ellos son los que emiten el token. Nuestra API se convierte en un "Resource Server" (Servidor de Recursos) puro: ya no guarda contraseñas, no tiene tabla de usuarios y no emite tokens; simplemente confía en las firmas

del servidor externo y valida el acceso. Justo por esto es que nuestra aplicación nunca ve la contraseña del usuario: porque el usuario la digita en la pantalla de Keycloak, dejándonos completamente libres de ese riesgo.

¿Dónde se ve en mi código?

La diferencia principal se nota en lo que desapareció de mi archivo SecurityConfig.java. Ya no tengo UserDetailsService (porque las tablas de base de datos ya no se usan para el login), no tengo un AuthController para iniciar sesión, ni llaves RSA privadas para firmar. Toda esta nueva arquitectura se ve en dos partes:

application.properties: Aquí está la línea más importante del proyecto: `spring.security.oauth2.resourceserver.jwt.issuer-uri=http://localhost:8090/realms/academy`. Con solo esta línea, Spring va automáticamente a esa URL, descarga la configuración de Keycloak y obtiene la llave pública para validar los tokens. No tuve que escribir nada de criptografía.

keycloakConverter (Traducción de roles): Keycloak no guarda los roles donde Spring los busca por defecto; los esconde dentro de un claim llamado "realm_access". En mi código implementé el método `extraerRoles` para entrar a ese JSON, sacar los roles y ponerles el prefijo `ROLE_`. Gracias a este traductor, mis reglas de autorización (`hasRole("EMPLOYEE")`, `hasRole("ADMIN")`) se quedaron idénticas a las del proyecto de HTTP Basic.

¿Qué pasa si no lo utilizo?

Si no uso OAuth2 y decido quedarme emitiendo mis propios JWTs, asumo toda la responsabilidad de la seguridad. Tendría que programar el reseteo de contraseñas, encriptar todo, proteger mi base de datos de filtraciones y rotar las llaves RSA. Sin OAuth2, mi sistema necesita mucho mantenimiento, y también es vulnerable a errores humanos en la gestión de identidad.

Cómo correrlo

Desde la terminal en la carpeta 03-security-oauth2:

Levanto el proyecto: `./mvnw spring-boot:run`

(Asumiendo que Keycloak ya está corriendo en Docker en el puerto 8090), la API arrancará en el puerto 8073, esta bloqueará peticiones vacías y aceptará solo tokens válidos emitidos por el realm "academy".

Nota: Sé que en un entorno de producción real el archivo `private.pem` jamás debe subirse a Git y debe inyectarse por variables de entorno, pero lo dejé en el repositorio para que el proyecto pueda correr sin pasos extra.